

# Technical Design Document: Chiến Lược Tái Cấu Trúc Toàn Diện AI Services Cho Hệ Thống FANG FastAPI V2

## 1. Bối Cảnh Lịch Sử Và Động Lực Nâng Cấp Hệ Thống Kiến Trúc FANG

Hệ thống FANG phiên bản 2 (v2) hiện đang hoạt động với tư cách là một lõi tri tuệ nhân tạo trung tâm (AI Core) độc lập, cung cấp giao diện lập trình ứng dụng (API) cho toàn bộ các ứng dụng ngoại vi (Thin Client) điển hình như miCareer-mini thông qua tập lệnh `fang_client.py` và giao diện người dùng Streamlit.<sup>1</sup> Sự phân tách này tuân thủ nguyên tắc thiết kế "Thick Core - Thin Client", trong đó toàn bộ gánh nặng về xử lý nghiệp vụ, tích hợp mô hình ngôn ngữ lớn (LLM), lưu trữ vector và điều phối RAG (Retrieval-Augmented Generation) được dồn hoàn toàn vào máy chủ FastAPI.<sup>1</sup> Các điểm cuối (endpoints) của hệ thống được cô lập dưới tiền tố định tuyến `/v2/`, với khả năng bảo vệ nguyên tắc chia sẻ tài nguyên nguồn gốc chéo (CORS) thông qua cấu hình `CORS_ALLOWED_ORIGINS` linh hoạt.<sup>1</sup>

Mặc dù kiến trúc v2 đã chứng minh được tính hiệu quả trong việc hỗ trợ các luồng xử lý phức tạp, điển hình là bộ phân giải tài liệu 5 lớp (5-Tier Parser) và truy vấn đa nguồn, cơ chế tích hợp AI Services hiện tại đang dựa trên nền tảng của các bộ chuyển đổi tự viết (Custom Adapters).<sup>1</sup> Tập tin `app/services/rag_model_adapters.py` đảm nhận vai trò kết nối với 5 loại mô hình từ ba nhà cung cấp lớn là Gemini (Google), OpenAI và Anthropic.<sup>1</sup> Cách tiếp cận này yêu cầu hệ thống phải trực tiếp nhập (import) và duy trì các bộ công cụ phát triển phần mềm (SDK) riêng lẻ, dẫn đến một khối lượng mã nguồn tương thích (boilerplate code) khổng lồ, đi kèm với logic xử lý ngoại lệ (try-catch) phân mảnh và khó kiểm thử tự động. Sự cứng nhắc này biểu hiện rõ nhất qua biến `MODEL_CANDIDATES`, nơi các cấu hình, giới hạn ngữ cảnh và ngân sách từ vựng bị mã hóa cứng (hardcoded) vào logic ứng dụng.<sup>1</sup>

Động lực chính thúc đẩy sự ra đời của thiết kế này bắt nguồn trực tiếp từ các quyết định mang tính chiến lược được phê duyệt sau giai đoạn nghiên cứu (R&D Phase 2, P1 & P2). Dựa trên các chỉ thị và xác nhận định hướng kỹ thuật từ người chịu trách nhiệm hệ thống (được ghi nhận là "Hưng"), ba giới hạn cốt lõi của kiến trúc hiện hành cần được tái thiết kế triệt để bao gồm:

Thứ nhất, đối với lớp nhúng dữ liệu không gian (Embedding Layer), hệ thống hiện đang lưu trữ vector với số chiều cố định 1024-dim dưới định dạng `halfvec(1024)` của công cụ `pgvector` trên cơ sở dữ liệu PostgreSQL.<sup>1</sup> Định dạng này tối ưu hóa việc sử dụng RAM và tăng tốc độ thuật toán HNSW cosine.<sup>1</sup> Việc thay đổi các mô hình nhúng (Embedding Models) trong tương lai có thể trả về các vector có kích thước không tương thích (ví dụ: 1536 chiều của mô hình nguyên bản `text-embedding-3-small` hoặc 768 chiều từ các mô hình mã nguồn mở).<sup>1</sup> Quyết định kỹ thuật tối cao được đưa ra là: Tuyệt đối tránh việc sửa đổi lược đồ cơ sở dữ liệu (Database

Schema Migration) sau khi hệ thống đã đi vào vận hành.<sup>1</sup> Chiến lược chuẩn hóa tại lớp ứng dụng thông qua kỹ thuật "Zero-Padding" (đệm không) và "Truncation" (cắt cụt) được chốt như một phương án phòng thủ vững chắc nhất.<sup>1</sup> Giải pháp này đóng vai trò như một cơ chế đệm, giữ cho hệ thống không bị phá vỡ cấu trúc và tạo tiền đề an toàn cho các chiến dịch nhúng lại toàn bộ dữ liệu (re-embed) trong tương lai khi hạ tầng cho phép.<sup>1</sup>

Thứ hai, để đạt được trạng thái "model-agnostic" (bất khả tri về mô hình), hệ thống cần một cơ chế chuẩn hóa giao tiếp đầu ra/đầu vào. Việc sử dụng các Cổng Trí tuệ Nhân tạo độc lập (AI Gateway như Kong hoặc Portkey) đã bị loại bỏ hoàn toàn khỏi tầm nhìn kiến trúc.<sup>1</sup> Nguyên nhân cốt lõi là do hạn chế về tài nguyên hệ thống của một dự án phát triển cá nhân/độc lập (Indie Developer); việc phải duy trì thêm các thùng chứa (containers) trung gian sẽ tạo ra gánh nặng vận hành quá mức (DevOps overhead) và tăng độ trễ mạng lưới nội bộ.<sup>1</sup> Thay vào đó, quyết định được thống nhất là nhúng trực tiếp bộ thư viện LiteLLM Python SDK vào trong lõi của FastAPI.<sup>1</sup> Thư viện này sẽ hoạt động như một lớp proxy cục bộ, tiêu chuẩn hóa mọi lệnh gọi API theo định dạng chung của OpenAI, biến mọi mô hình từ các nhà cung cấp khác nhau trở thành một giao diện đồng nhất.<sup>1</sup>

Thứ ba, sự dịch chuyển từ cấu hình tĩnh sang Cấu hình Động Tập trung (Centralized Dynamic Registry).<sup>1</sup> Logic dự phòng mô hình (Fallback Delegation) và kiểm soát định tuyến hiện đang nằm rải rác ở rag\_orchestrator.py và rag\_model\_adapters.py sẽ bị triệt tiêu hoàn toàn.<sup>1</sup> Toàn bộ quyền quyết định mô hình nào được gọi, với ngân sách bao nhiêu, chi phí ra sao và chuỗi dự phòng là gì sẽ được nhường lại cho một tệp tin cấu hình duy nhất: models\_registry.yaml.<sup>1</sup> Tệp tin này đóng vai trò là "Nguồn chân lý duy nhất" (Single Source of Truth), tách biệt hoàn toàn mã nguồn nghiệp vụ khỏi dữ liệu cấu hình môi trường.<sup>1</sup>

Báo cáo thiết kế kỹ thuật này sẽ phác thảo chi tiết cách thức mã nguồn Python, kiến trúc thư mục, hệ thống luồng RAG và luồng nạp dữ liệu (Ingestion Pipeline) được định hình lại để phản ánh chính xác các quyết định chiến lược đã nêu trên.

## 2. Phân Tích So Sánh Trạng Thái Hệ Thống: As-Is So Với To-Be

Để làm rõ lộ trình chuyển đổi và giá trị mang lại từ đợt tái cấu trúc, việc đối chiếu giữa kiến trúc hiện hành và kiến trúc đích là bước thiết yếu. Bảng dưới đây cung cấp một cái nhìn toàn diện về sự dịch chuyển trong các thành phần cốt lõi của lõi AI FANG.

| Tiêu Chí So Sánh | Trạng Thái Hiện Tại (As-Is - FANG v2 Core) | Trạng Thái Đích Đề Xuất (To-Be - FANG AI Services Refactored) |
|------------------|--------------------------------------------|---------------------------------------------------------------|
|                  |                                            |                                                               |

|                                            |                                                                                                                                                    |                                                                                                                                                         |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Quản Lý Giao Tiếp Mô Hình</b>           | Phụ thuộc vào app/services/rag_model_adapters.py chứa nhiều logic kiểm tra cấu trúc SDK độc quyền (Google google-generativeai, OpenAI, Anthropic). | Xóa bỏ hoàn toàn lớp bộ chuyển đổi tự viết. Sử dụng litellm.completion() chuẩn hóa mọi tương tác ngay trong mã nguồn FastAPI.                           |
| <b>Kiểm Soát Cấu Hình</b>                  | Khai báo biến hằng số MODEL_CANDIDATES ngay trong tệp Python, khó khăn trong việc thiết lập A/B testing hoặc đổi model khi đang chạy (hot-swap).   | "Nguồn chân lý duy nhất" thông qua tệp app/core/models_registry.yaml, hỗ trợ phân tầng chi phí và giới hạn ngân sách ngữ cảnh độc lập.                  |
| <b>Cơ Chế Phục Hồi Lỗi (Fallback)</b>      | Mã nguồn vòng lặp try...except lồng ghép thủ công trong rag_orchestrator.py để leo thang từ mô hình này sang mô hình khác.                         | Sử dụng tính năng Ủy quyền Dự phòng tự động của LiteLLM. Ứng dụng chỉ truyền tham số fallbacks=["model_a", "model_b"] vào hàm.                          |
| <b>Quản Lý Xử Lý Ngoại Lệ (Exceptions)</b> | Mỗi SDK sinh ra một loại lỗi riêng (ví dụ: Google API Error khác với OpenAI Rate Limit). Hệ thống phải bắt (catch) và ánh xạ từng loại lỗi.        | LiteLLM chuẩn hóa thành các lớp ngoại lệ thống nhất (litellm.exceptions.RateLimitError, APIError). Lớp dịch vụ xử lý đồng nhất và tinh gọn.             |
| <b>Bảo Vệ Kiến Trúc Lưu Trữ Vector</b>     | Schema cố định halfvec(1024). Nếu text-embedding-3-small hoặc model thay thế trả về chiều dài khác, app/services/persistence.py                    | Tích hợp lớp chuẩn hóa (Zero-Padding / Truncation) sử dụng Numpy trong bộ nhớ ứng dụng. Vector luôn được đúc về đúng 1024 chiều trước khi vào Postgres. |

|                                                |                                                                                                                                                          |                                                                                                                                                                      |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                | sẽ gặp lỗi tràn cơ sở dữ liệu.                                                                                                                           |                                                                                                                                                                      |
| <b>Kiểm Soát Chất Lượng Phân Giải (Parser)</b> | Cổng chất lượng (ProTierGate) lồng ghép chung với cơ chế thử lại của mạng, làm mờ ranh giới giữa lỗi kỹ thuật và lỗi nghiệp vụ.                          | Tách bạch hoàn toàn: LiteLLM quản lý lỗi mạng lưới hạ tầng. ProTierGate chỉ tập trung phân tích văn bản AI (heuristic) xem chất lượng có đạt yêu cầu JSON không.     |
| <b>Cơ Chế Theo Dõi Ngân Sách Hội Thoại</b>     | Trích xuất bộ đếm token rườm rà. Hệ thống phải đếm thủ công dựa trên thư viện tiktoken hoặc ước lượng không chuẩn xác trên các mô hình Anthropic/Gemini. | LiteLLM hỗ trợ stream_options={"include_usage": True}. Phân giải tự động prompt_tokens cho mọi dòng dữ liệu (stream), kích hoạt cảnh báo 80% độ chính xác tuyệt đối. |

Sự chuyển dịch này không chỉ giải quyết bài toán giảm thiểu nợ kỹ thuật (technical debt) mà còn gia tăng đáng kể khả năng đáp ứng của hệ thống trước sự biến thiên khó lường của thị trường cung cấp dịch vụ AI đám mây, tuân thủ đúng nguyên lý Đóng-Mở (Open-Closed Principle) trong kỹ nghệ phần mềm.

### 3. Kiến Trúc Cấu Hình Động Tập Trung (Centralized Dynamic Registry)

Trọng tâm của bản thiết kế nằm ở khả năng tách rời siêu dữ liệu của mô hình ra khỏi hệ thống thực thi. Sự ra đời của tệp tin models\_registry.yaml là bước tiến then chốt, tạo ra một kho lưu trữ thông tin tập trung, mô tả không chỉ danh tính của các mô hình khả dụng mà còn là quy tắc định tuyến, ngân sách, chuỗi dự phòng và tham số nhúng.<sup>1</sup>

#### 3.1. Thiết Kế Lược Đồ Tập models\_registry.yaml

Cấu trúc YAML được thiết kế để phản ánh hệ sinh thái 5 lớp hiện hành của mạng lưới FANG.<sup>1</sup> Nhóm mô hình được chia thành hai phân vùng rõ rệt: Nhóm Lite (Tiers 1-3) đóng vai trò tiền tuyến với chi phí siêu rẻ và tốc độ phản hồi tính bằng mili-giây, phục vụ các truy vấn sàng lọc đơn giản và phân giải tài liệu<sup>1</sup>; và Nhóm Pro (Tiers 4-5) đóng vai trò vũ khí chiến lược, giải quyết các nhiệm vụ đòi hỏi khả năng tư duy logic và lập luận sâu (complex reasoning), dù chi phí có

thể cao gấp 5 đến 20 lần.<sup>1</sup>

Bên cạnh đó, nó quy định rõ ràng lược đồ nhúng không gian. Do cấu hình cục bộ nhắm đến môi trường phát triển (DEV) với số chiều mặc định là 1024 và tối ưu hóa tài nguyên phần cứng, các đặc tả kỹ thuật này phải được ghi nhận rõ ràng.<sup>1</sup> Dưới đây là kiến trúc tệp models\_registry.yaml chi tiết:

## YAML

```
# Đường dẫn hệ thống: app/core/models_registry.yaml
# Phiên bản định dạng: 1.0.0
# Ngày phê duyệt cập nhật: 10/05/2026
# Mục đích: Nguồn chân lý duy nhất (Single Source of Truth) điều phối toàn bộ AI Services

metadata:
  description: "Centralized Dynamic Registry for FANG FastAPI AI Layer."
  maintainer: "FANG Systems Architect"

# =====
# 1. CẤU HÌNH LỚP NHÚNG KHÔNG GIAN (EMBEDDING LAYER CONFIGURATION)
# =====
embedding_models:
  default_provider: "openai"
  target_dimension: 1024 # Quyết định chiến lược: Bắt buộc đúc vector về 1024 chiều để bảo vệ
pgvector
models:
  text-embedding-3-small:
    litellm_path: "openai/text-embedding-3-small"
    native_dimension: 1536 # Kích thước nguyên bản từ nhà cung cấp
    cost_per_1m_tokens: 0.02
    description: "Default embedding model for DEV environment. Requires truncation down to 1024-dim."
  open-source-768:
    litellm_path: "huggingface/bge-base-en-v1.5"
    native_dimension: 768
    cost_per_1m_tokens: 0.00
    description: "Future alternative. Requires zero-padding up to 1024-dim."

# =====
# 2. ĐỊNH NGHĨA MÔ HÌNH SINH VĂN BẢN (GENERATION MODELS)
# =====
generation_models:
  # LITE TIERS: Tối ưu cho tốc độ và giá thành. Được ưu tiên chạy trước.
```

```
gemini-flash:
  tier: 1
  litellm_path: "gemini/gemini-3-flash"
  context_window: 800000
  cost_per_1m_input: 0.35
  cost_per_1m_output: 1.05
  description: "Tier 1: Lite fallback origin. Primary for heavy CV ingestion pipelines."
```

```
gpt-mini:
  tier: 2
  litellm_path: "openai/gpt-5.4-mini"
  context_window: 128000
  cost_per_1m_input: 0.15
  cost_per_1m_output: 0.60
  description: "Tier 2: Primary fallback for Tier 1 failures."
```

```
claude-haiku:
  tier: 3
  litellm_path: "anthropic/claude-4.5-haiku"
  context_window: 200000
  cost_per_1m_input: 0.25
  cost_per_1m_output: 1.25
  description: "Tier 3: The final safety net for the Lite model ecosystem."
```

```
# PRO TIERS: Tối ưu cho lập luận phức tạp. Chỉ gọi khi vượt qua ProTierGate.
```

```
gemini-pro:
  tier: 4
  litellm_path: "gemini/gemini-pro"
  context_window: 1000000
  cost_per_1m_input: 3.50
  cost_per_1m_output: 10.50
  description: "Tier 4: Entry point for complex reasoning tasks."
```

```
gpt-full:
  tier: 5
  litellm_path: "openai/gpt-5.5"
  context_window: 128000
  cost_per_1m_input: 5.00
  cost_per_1m_output: 15.00
  description: "Tier 5: Ultimate model. Highest cost, highest capability."
```

```
# =====
# 3. CHÍNH SÁCH ĐỊNH TUYẾN VÀ CHUỖI DỰ PHÒNG (ROUTING MODES & FALLBACKS)
```

```
# =====
routing_modes:
# Các chiến lược dự phòng tự động nhiều cấp
auto-lite:
primary: "gemini-flash"
fallbacks: ["gpt-mini", "claude-haiku"]
description: "Standard mode for parsing and basic RAG. Cascades through tiers 1, 2, and 3."

auto-pro:
primary: "gemini-pro"
fallbacks: ["gpt-full"]
description: "Escalation mode activated via ProTierGate. Cascades through tiers 4 and 5."

# Các chiến lược chỉ định trực tiếp (Không có Fallbacks để tránh tiêu hao tài nguyên ngoài ý muốn)
direct-gpt-mini:
primary: "gpt-mini"
fallbacks:
description: "Strictly calls GPT Mini. Used when HR user explicitly overrides auto mode."

direct-gemini-pro:
primary: "gemini-pro"
fallbacks:
description: "Strictly calls Gemini Pro for user-forced complex reasoning."
```

### 3.2. Cơ Sở Lập Luận Thiết Kế Khai Báo (Declarative Rationale)

1. **Tiền Tố Định Tuyển Chuẩn Hóa (litellm\_path):** Thuộc tính này là chìa khóa để khai thác LiteLLM. Bằng cách nối tên nhà cung cấp với tên mô hình (ví dụ: anthropic/claude-4.5-haiku hoặc gemini/gemini-3-flash)<sup>1</sup>, cấu hình này cung cấp cho litellm.completion() điểm kết nối đích chính xác mà không yêu cầu hệ thống phải biên dịch lại logic nội bộ.<sup>1</sup> Bất kỳ thay đổi phiên bản mô hình nào trong tương lai (như việc nâng cấp từ claude-4.5-haiku lên một phiên bản tiềm năng claude-5) chỉ yêu cầu sửa đổi văn bản YAML.
2. **Đại lượng context\_window (Giới hạn Ngân sách Từ vựng):** Tham số này được sử dụng làm mẫu số cho phép tính toán cảnh báo ngữ cảnh tại bước thứ 8 trong luồng 12 bước của hệ thống truy vấn đàm thoại RAG.<sup>1</sup> Do chiến lược v2 đã loại bỏ cơ chế "Trượt Khung Ngữ Cảnh" (Sliding Window) nhằm bảo tồn thông tin, hệ thống phải liên tục kiểm tra tổng số lượng token đã tiêu hao so với giới hạn của mô hình.<sup>1</sup> Cấu hình tĩnh này cho phép FastAPI lấy context\_window (ví dụ: 800,000 cho Gemini Flash) nhân với ngưỡng CONTEXT\_BUDGET\_WARNING\_THRESHOLD (thường là 80%) để đưa ra cảnh báo contextWarning khi phiên làm việc phình to.<sup>1</sup>
3. **Khóa routing\_modes với Mạng fallbacks:** Cấu trúc này ánh xạ tỷ lệ 1:1 với khái niệm 7 chế độ modelMode từ hệ thống phiên bản trước.<sup>1</sup> Đặc biệt, nó loại bỏ khối mã if-else

khổng lồ. Nếu tham số fallbacks trống, ứng dụng sẽ hiểu rằng người dùng nhân sự (HR) đã chọn đích danh một mô hình duy nhất và từ chối kích hoạt chuyển đổi chuỗi dự phòng âm thầm, đáp ứng chính xác yêu cầu về tính minh bạch đã định nghĩa trong tài liệu quy trình truy vấn hệ thống.<sup>1</sup>

### 3.3. Giải Pháp Mã Nguồn Nạp Cấu Hình Động Trình Quản Lý Dữ Liệu (Registry Manager)

Để hiện thực hóa tệp YAML vào bộ nhớ ứng dụng FastAPI, lớp RegistryManager được xây dựng dựa trên mẫu thiết kế Singleton kết hợp với thư viện Pydantic. Việc sử dụng Pydantic đảm bảo tính toàn vẹn và an toàn kiểu dữ liệu (type safety), một nguyên tắc bắt buộc trong các hệ thống phần mềm doanh nghiệp cấp sản xuất (production-grade software).

Python

```
# app/core/models/registry_schema.py
import yaml
import logging
from typing import Dict, List, Optional
from pydantic import BaseModel, Field, field_validator

logger = logging.getLogger(__name__)

# --- Định nghĩa Schema Pydantic Xác thực Siêu Dữ Liệu ---

class EmbeddingModelConfig(BaseModel):
    litellm_path: str = Field(..., description="Đường dẫn định tuyến chuẩn của LiteLLM")
    native_dimension: int = Field(..., description="Số chiều gốc của mô hình")
    cost_per_1m_tokens: float = Field(default=0.0)
    description: Optional[str] = None

class EmbeddingRegistry(BaseModel):
    default_provider: str = Field(..., description="Nhà cung cấp nhúng được ưu tiên")
    target_dimension: int = Field(1024, description="Kích thước không gian ép buộc chuẩn hóa (Zero-padding/Truncation)")
    models: Dict[str, EmbeddingModelConfig]

    @field_validator('target_dimension')
    def enforce_1024_dimension(cls, v):
        if v != 1024:
            logger.warning(f"Target dimension {v} differs from typical 1024-dim setup.")
```



```
return v
```

```
class GenerationModelConfig(BaseModel):
```

```
    tier: int = Field(ge=1, le=5, description="Cấp độ từ 1 (Lite) đến 5 (Pro)")
```

```
    litellm_path: str = Field(..., description="Đường dẫn gọi LiteLLM")
```

```
    context_window: int = Field(gt=0, description="Giới hạn số token tối đa cho hội thoại")
```

```
    cost_per_1m_input: float
```

```
    cost_per_1m_output: float
```

```
    description: Optional[str] = None
```

```
class RoutingModeConfig(BaseModel):
```

```
    primary: str = Field(..., description="Tên mô hình chính (phải ánh xạ với generation_models)")
```

```
    fallbacks: List[str] = Field(default_factory=list, description="Danh sách tuần tự các mô hình dự phòng")
```

```
    description: Optional[str] = None
```

```
class ModelRegistrySchema(BaseModel):
```

```
    metadata: Dict[str, str]
```

```
    embedding_models: EmbeddingRegistry
```

```
    generation_models: Dict[str, GenerationModelConfig]
```

```
    routing_modes: Dict
```

```
# --- Lớp Trình Quản Lý Mẫu Độc Bản (Singleton Manager) ---
```

```
class RegistryManager:
```

```
    """
```

```
    Trình quản lý cấu hình AI nội bộ cung cấp giao diện truy xuất Nguồn Chân Lý Duy Nhất.
```

```
    """
```

```
    _instance = None
```

```
    _config: Optional = None
```

```
def __new__(cls, filepath: str = "app/core/models_registry.yaml"):
```

```
    if cls._instance is None:
```

```
        cls._instance = super(RegistryManager, cls).__new__(cls)
```

```
        cls._instance._load_and_validate(filepath)
```

```
    return cls._instance
```

```
def _load_and_validate(self, filepath: str) -> None:
```

```
    try:
```

```
        with open(filepath, 'r', encoding='utf-8') as file:
```

```
            raw_yaml = yaml.safe_load(file)
```

```
            # Ép kiểu và kiểm định tự động thông qua Pydantic
```

```
            self._config = ModelRegistrySchema(**raw_yaml)
```

```

        logger.info("Successfully loaded and validated Centralized Dynamic Registry.")
    except FileNotFoundError:
        logger.error(f"Critical System Error: YAML config not found at {filepath}")
        raise
    except Exception as e:
        logger.error(f"Schema validation failed for AI Registry: {str(e)}")
        raise

    @classmethod
    def get_config(cls) -> ModelRegistrySchema:
        if cls._config is None:
            raise RuntimeError("RegistryManager is not initialized. Ensure app startup event calls load().")
        return cls._config

```

Quá trình khởi tạo lớp RegistryManager được đăng ký vào chu kỳ khởi động của FastAPI (Lifespan event trong app/main.py), đảm bảo rằng toàn bộ thông số định tuyến và cấu hình mạng lưới đã được nạp vào RAM trước khi nhận bất kỳ yêu cầu kết nối nào từ phía các lớp dịch vụ (Client Layer). Bất kỳ thay đổi thông số nào không hợp lệ đều sẽ gây ra lỗi tại thời điểm hệ thống bắt đầu chạy (fail-fast), một đặc tính vô cùng quan trọng để ngăn chặn lỗi hệ thống tinh lan truyền trong suốt quá trình chạy.

## 4. Tái Thiết Kế Luồng Điều Phối Sinh Văn Bản (Generation Services & RAG Orchestration)

Một trong những quy trình phức tạp nhất trong kiến trúc FANG là luồng truy vấn đàm thoại qua 12 bước chi tiết được điều khiển bởi tệp app/services/rag\_orchestrator.py và app/services/rag\_query.py. Luồng thông tin bắt đầu từ việc xác thực ứng viên (bước 1), nạp lịch sử tương tác đàm thoại không dùng khung trượt (bước 7), lắp ghép ngữ cảnh đa nguồn (bước 6), trước khi thực hiện bước thứ 10 là Gọi mô hình ngôn ngữ (LLM Invocation) kết hợp chính sách retry dự phòng và kết thúc tại việc duy trì hệ thống kiểm toán lưu trữ (Bước 11 & 12).<sup>1</sup>

Sự xuất hiện của công nghệ LiteLLM thay đổi hoàn toàn cục diện cách khối điều phối (Orchestrator) được triển khai. Nó đảm nhận vai trò quản lý vòng đời (lifecycle management) đối với các lệnh gọi, đồng thời kết xuất dữ liệu đo lường theo dõi tài nguyên ngân sách.

### 4.1. Cơ Chế Ủy Quyền Dự Phòng Tự Động (Fallback Delegation)

Trước đây, khi nhóm hệ thống gọi một chế độ auto-lite, một quy trình được phân rã thủ công thông qua 3 khối Try-Catch phải được cấu trúc để truyền tải liệu cho Gemini Flash (Tier 1), nếu thất bại với các lỗi API quá tải (Rate limit), nó phải tự khởi tạo lại đường truyền đến phiên bản GPT-mini (Tier 2), và kế tiếp là Claude Haiku (Tier 3).<sup>1</sup> Với nguyên lý thiết kế hệ thống LiteLLM, nhà phát triển cấu trúc lại việc định tuyến này thông qua một thông số danh sách thuần túy (tham số fallbacks=...).<sup>1</sup> Mã nguồn xử lý lỗi của khối Điều Phối Trí Tuệ RAG (RAG Orchestrator)

được làm sạch triệt để.

Dưới đây là một ví dụ chuyên sâu về việc thiết kế lớp RAGOrchestrator trong FastAPI với vai trò tiêu chuẩn hóa mô hình đàm thoại đa nhà cung cấp, với chú giải cho các quyết định kiến trúc:

Python

```
# app/services/rag_orchestrator.py
import logging
from typing import List, Dict, Any
from litellm import completion
from litellm.exceptions import RateLimitError, APIError, ContextWindowExceededError
from app.core.models.registry_schema import RegistryManager
from app.core.config import settings # Lưu trữ hằng số CONTEXT_BUDGET_WARNING_THRESHOLD

logger = logging.getLogger(__name__)

class RAGOrchestrator:
    """
    Trái tim của hệ thống Luồng Truy Vấn Đàm Thoại RAG (Bước 10).
    Đảm nhận việc quản lý ngữ cảnh đàm thoại, gọi thư viện LiteLLM,
    ủy quyền dự phòng và kiểm tra ngân sách từ vựng.
    """
    def __init__(self):
        # Lấy tham chiếu siêu dữ liệu tĩnh từ RAM
        self.registry = RegistryManager.get_config()
        self.warning_threshold = getattr(settings, 'CONTEXT_BUDGET_WARNING_THRESHOLD', 0.8)

    def invoke_generation(
        self,
        assembled_messages: List,
        model_mode: str,
        stream: bool = True
    ) -> Dict[str, Any]:
        """
        Gửi yêu cầu tới nền tảng LLM theo chế độ model_mode xác định
        (ví dụ: 'auto-lite', 'direct-gpt-mini', 'auto-pro').
        """
        # Bước 10.1: Phân giải định tuyến từ Nguồn Chân Lý
        if model_mode not in self.registry.routing_modes:
            raise ValueError(f"Hệ thống không nhận dạng model_mode: {model_mode}. Kiểm tra yaml.")
```

```

routing_cfg = self.registry.routing_modes[model_mode]
primary_key = routing_cfg.primary

# Tra cứu tiền tố chuẩn (ví dụ: 'gemini/gemini-3-flash')
primary_model_path = self.registry.generation_models[primary_key].litellm_path

# Ánh xạ các key dự phòng thành các đường dẫn tương ứng
fallback_paths = [
    self.registry.generation_models[fb_key].litellm_path
    for fb_key in routing_cfg.fallbacks
]

logger.info(f"Khởi động RAG Orchestrator. Tuyến chính: {primary_model_path}. Dự phòng: {fallback_paths}")

try:
    # Tham số then chốt: stream_options={"include_usage": True}
    # Yêu cầu bắt buộc để bước 8 (Budget Check) có dữ liệu định lượng
    stream_config = {"include_usage": True} if stream else None

    # Bước 10.2: Ủy quyền Dự phòng (Fallback Delegation) vào LiteLLM SDK
    response = completion(
        model=primary_model_path,
        messages=assembled_messages,
        fallbacks=fallback_paths if fallback_paths else None,
        stream=stream,
        stream_options=stream_config
    )

    # Nếu phản hồi theo dạng luồng (stream), hệ thống FastAPI Generator sẽ
    # chịu trách nhiệm đọc và trả các chunk, đồng thời phân tích đối tượng usage ở mảnh cuối cùng.
    if stream:
        return {"type": "streaming_response", "data": response, "model_key": primary_key}
    else:
        # Phục vụ luồng phân giải CV Parser (Không stream)
        return {
            "type": "static_response",
            "content": response.choices.message.content,
            "usage": response.usage.model_dump() if response.usage else None,
            "model_key": primary_key
        }

# Xử lý các lỗi được LiteLLM chuẩn hóa thay cho lỗi phân mảnh của SDK nhà cung cấp

```

```

except RateLimitError as e:
    logger.error(f"Khóa tài nguyên cạn kiệt (Rate Limit Exhausted) trên toàn bộ chuỗi Fallback: {str(e)}")
    raise RuntimeError("Hệ thống AI hiện đang quá tải. Xin vui lòng thử lại sau ít phút.")

except ContextWindowExceededError as e:
    logger.error(f"Tràn khung ngữ cảnh: {str(e)}")
    raise ValueError("Kích thước hội thoại đã vượt quá năng lực mô hình. Đề xuất tạo hội thoại mới.")

except APIError as e:
    logger.error(f"Lỗi cổng giao tiếp trung tâm (Upstream Gateway Failure): {str(e)}")
    raise SystemError("Tuyến kết nối hệ thống AI bị gián đoạn.")

```

Sự loại bỏ của `rag_model_adapters.py` là một chiến thắng thiết kế đáng kể.<sup>1</sup> Mã nguồn không còn chứa các lệnh `import google.generativeai` hay `import anthropic`. Lớp `RAGOrchestrator` trở nên tinh gọn, thuần túy là một cơ sở hạ tầng mạng nhận dữ liệu từ các khối đa nguồn (`JobPosting`, `Candidate Profile`, `Retrieved CV Chunks`)<sup>1</sup> và điều phối thông qua chức năng duy nhất `litellm.completion()`.<sup>1</sup> Khả năng tương thích nền tảng, cơ chế retry khi trễ mạng và phân tích chuỗi văn bản bị đứt gãy hoàn toàn được quản lý tự động bởi LiteLLM.

## 4.2. Giải Quyết Vấn Đề Quản Lý Ngân Sách Khung Ngữ Cảnh

Bước 8 trong chu trình là Kiểm tra Ngân sách (Budget Check). Bối cảnh đặc thù của hệ thống RAG đàm thoại cho nhân sự HR tại FANG là việc ghép nối một số lượng dữ liệu lớn tại bước 6: "Context Assembly" (Lắp ghép Ngữ Cảnh).<sup>1</sup> System Prompt chuyên dụng được chèn hàng loạt các khối văn bản (blocks) thông tin: , , và đặc biệt là (nội dung bình duyệt nhiều vòng từ interviewer).<sup>1</sup>

Sự phình to của chuỗi System Prompt, kết hợp với thực tế v2 loại bỏ cơ chế "Sliding Window" (Xóa bớt lịch sử cũ) tại bước 7 nhằm giữ vững dòng suy luận của AI<sup>1</sup>, dẫn đến một rủi ro hiện hữu: Tràn giới hạn token của mô hình (ví dụ Claude Haiku chỉ đạt 200,000 max context).<sup>1</sup>

Đoạn mã cấu hình `stream_options={"include_usage": True}` từ LiteLLM SDK đóng vai trò sống còn trong chiến lược bảo vệ hệ thống.<sup>1</sup> Khi API FastAPI của lõi thực thi việc tiêu thụ đoạn văn bản theo luồng (chunk streaming), đối tượng cuối cùng được trả về sẽ mang theo gói dữ liệu đo lường thông số token (`prompt_tokens + completion_tokens`).

FastAPI tại điểm cuối (Endpoint) sẽ đọc thông số này, đối chiếu với chỉ số `context_window` thuộc về `model_key` đang thao tác đã cấu hình trong `models_registry.yaml`. Phép toán logic được kích hoạt:

$$\text{Tính Tổng Mức Tiêu Thụ (\%)} = \left( \frac{\text{Tổng Token Mới Nhất}}{\text{Giới hạn } context\_window} \right) * 100$$

Nếu giá trị vượt quá hằng số hệ thống `CONTEXT_BUDGET_WARNING_THRESHOLD` (mặc định

80%), tầng Middleware của lớp Dịch Vụ sẽ tự động đẩy một thẻ cảnh báo contextWarning vào một kênh phụ trong kiến trúc kết xuất phía Web.<sup>1</sup> Cảnh báo này cung cấp cho người dùng nhân sự hai tùy chọn xử lý khẩn cấp: (1) "Tóm tắt & tiếp tục" (hệ thống sẽ tự động gửi toàn bộ văn bản cho một mô hình Tier 1 giá cực rẻ để nén lại trước khi tái khởi động chuỗi RAG); hoặc (2) "Bắt đầu hội thoại mới".<sup>1</sup> Phương pháp Tóm tắt-Thay thế (Token Budget + Summarization) này giải quyết được điểm yếu chí mạng của hệ thống Sliding Window cũ là đánh mất các dữ kiện nền của phiên phỏng vấn đầu tiên.<sup>1</sup>

## 5. Tái Kiến Trúc Đường Ống Xử Lý Đầu Vào (Ingestion Flow) & ProTierGate

Quy trình nạp dữ liệu ứng viên (Ingestion Flow) với mục đích biến một tệp tài liệu PDF lý lịch trích ngang (CV) hỗn loạn thành tập hợp các vector tri thức là luồng API quan trọng bậc nhất, được khai báo thông qua phương thức POST /v2/ingestion/jobs.<sup>1</sup> Lộ trình dữ liệu trải qua 6 công đoạn khép kín: Nhận sự kiện (Trigger) chứa tải trọng jobId và URL của CV (cvSnapUrl), chuyển đổi PDF sang dạng JSON thông qua cơ chế Fallback phân cấp (5-Tier), chuyển đổi sang kết cấu Markdown, phân mảnh văn bản hỗn hợp (Hybrid Chunking), Nhúng Vector (Embedding) và cuối cùng là Lưu trữ (Persistence).<sup>1</sup>

Giai đoạn phức tạp và tiêu hao chi phí lớn nhất nằm ở bước thứ 2: Bộ Phân Giải 5 Cấp Độ (5-Tier Parser).<sup>1</sup> Tương tự như hệ thống RAG đàm thoại, bộ phân giải này tận dụng năng lực phân cấp mô hình nhưng tuân thủ một bộ quy tắc khắt khe hơn: Nó dựa vào Cổng Phân Loại Nâng Cao (ProTierGate).<sup>1</sup>

### 5.1. Triết Lý Thiết Kế Của Cổng Phân Loại ProTierGate

Sự phân nhóm giữa Lite Tiers (Gemini Flash, GPT-mini, Claude Haiku) và Pro Tiers (Gemini Pro, GPT-5.5) là ranh giới của bài toán tối ưu chi phí (Cost Optimization). Ưu điểm của nhóm Lite là chi phí vận hành siêu thấp và tốc độ trả kết quả nhanh, vô cùng phù hợp cho các luồng phân giải CV đại trà.<sup>1</sup> Tuy nhiên, các CV có kết cấu đa cột, định dạng phi truyền thống hoặc cấu trúc câu trúc trắc thường gây nhầm lẫn cho khả năng thiết lập JSON của Lite models. Khi đó, nhóm mô hình Pro - với giá trị kinh tế có thể cao gấp 5 đến 20 lần chi phí vận hành - cần được huy động bởi khả năng lập luận đa không gian mạnh mẽ.<sup>1</sup>

Chốt kiểm soát chất lượng (Quality Gate) ProTierGate được cấu thành bởi các thuật toán chẩn đoán tự động (Heuristic Analysis). Theo các nguyên lý thiết kế, FANG thực thi quy định "Chỉ leo lên tầng cao cấp khi sai phạm thuộc về chất lượng, không leo thang nếu nguyên nhân do hạ tầng mạng".<sup>1</sup> Nghĩa là, nếu toàn bộ 3 tầng cấp Lite đều trả về RateLimitError hoặc Timeout, hệ thống sẽ bảo lưu trạng thái hệ thống, trả về lỗi hạ tầng và không kích hoạt mô hình Pro nhằm ngăn việc tiêu hao hàng chục đô-la vào một trung tâm dữ liệu đang bảo trì.<sup>1</sup> Ngược lại, nếu văn bản JSON trả về bị ngắn bất thường (dưới 50 ký tự) hoặc chứa các câu từ chối phục vụ (ví dụ: "Tôi không có thông tin", "Tôi không thể"), hệ thống sẽ gán cờ low\_confidence\_output và chuyển tiếp luồng yêu cầu cho nhóm Pro.<sup>1</sup>

## 5.2. Chuyển Đổi Logic Bộ Phân Giải Với LiteLLM

Cơ chế ủy quyền tự động của cấu hình định tuyến thông qua tệp YAML (auto-lite với 3 cấp độ dự phòng) đảm nhiệm toàn bộ phần quản lý mạng lưới (Network Reliability), giải phóng ứng dụng FastAPI khỏi các vòng lặp xử lý lỗi vô ích. Khối lượng mã nguồn nghiệp vụ trong `app/services/cv_parser.py` nay chỉ còn lại một giao diện kiểm soát Heuristic gọn gàng.<sup>1</sup>

Python

```
# app/services/cv_parser.py
import json
import logging
from app.services.rag_orchestrator import RAGOrchestrator

logger = logging.getLogger(__name__)

class QualityGateException(Exception):
    """Ngoại lệ chuyên dụng đánh dấu kết quả trả về từ mô hình có chất lượng thấp."""
    pass

class CVParserService:
    """
    Điều phối Bước 2 trong luồng Ingestion Flow: Chuyển hóa CV nguyên bản sang JSON.
    Thiết kế tận dụng tối đa kiến trúc phân tách ProTierGate.
    """
    def __init__(self):
        self.orchestrator = RAGOrchestrator()

    def _quality_gate_passed(self, raw_text: str) -> bool:
        """
        Thực thi quy tắc đánh giá Heuristic:
        1. Độ dài văn bản phải lớn hơn 50 ký tự.
        2. Không chứa các cụm từ chối trả lời do lớp lọc an toàn của LLM kích hoạt.
        """
        if not raw_text or len(raw_text.strip()) < 50:
            logger.warning("Quality Gate Failed: Text length under 50 characters.")
            return False

        denial_phrases = ["tôi không thể", "không có thông tin", "i cannot", "unable to process"]
        lower_text = raw_text.lower()
        if any(phrase in lower_text for phrase in denial_phrases):
```

```
logger.warning("Quality Gate Failed: Denial signal detected in output.")
return False
```

```
return True
```

```
def _format_to_json(self, raw_text: str) -> dict:
    """Làm sạch và ép kiểu chuỗi văn bản về JSON Object."""
    # Bỏ qua các dấu code block (```json...```) để parse
    clean_text = raw_text.replace("```json", "").replace("```", "").strip()
    try:
        return json.loads(clean_text)
    except json.JSONDecodeError:
        raise QualityGateException("JSON structure corrupted.")
```

```
def parse_cv_to_json(self, cv_raw_text: str) -> dict:
    """
    Giao diện chính phục vụ việc phân giải (Parsing) sử dụng chiến lược 5-Tier Fallback.
    """
    messages =
```

```
    try:
        # GIAI ĐOẠN 1: Tiến hành chuỗi Fallback Nội bộ Khối Lite (Flash -> Mini -> Haiku)
        # Cơ chế ủy quyền mạng lưới LiteLLM thực hiện tuần tự và tự động trả kết quả.
        logger.info("Executing Auto-Lite fallback chain for Ingestion.")
        response_lite = self.orchestrator.invoke_generation(
            assembled_messages=messages,
            model_mode="auto-lite",
            stream=False
        )
```

```
    output_content = response_lite.get("content", "")
```

```
    # GIAI ĐOẠN 2: Thử nghiệm Cổng Kiểm Soát Chất Lượng ProTierGate
    if self._quality_gate_passed(output_content):
        return self._format_to_json(output_content)
    else:
        # Kích hoạt leo thang thông qua cờ chất lượng
        raise QualityGateException("low_confidence_output")
```

```
except QualityGateException as qg_err:
    # GIAI ĐOẠN 3: Leo thang chiến lược (Escalation) sang chuỗi Pro Tiers
    # Nguyên nhân: Nhóm Lite xử lý được nhưng chất lượng ngữ nghĩa quá kém.
    # Chuỗi tự động kích hoạt: Gemini Pro -> GPT 5.5
```



```

        logger.warning(f"ProTierGate activated due to {str(qg_err)}. Escalating to auto-pro chain.")
        response_pro = self.orchestrator.invoke_generation(
            assembled_messages=messages,
            model_mode="auto-pro",
            stream=False
        )

        output_pro_content = response_pro.get("content", "")
        if self._quality_gate_passed(output_pro_content):
            return self._format_to_json(output_pro_content)
        else:
            # Ngay cả Pro Tiers cũng thất bại trong việc thiết lập dữ liệu
            raise RuntimeError("Luồng xử lý từ chối phân giải. CV quá phức tạp hoặc định dạng dị thường.")

    except Exception as e:
        # Nếu gặp sự cố hạ tầng (như APIError hoặc RateLimitError) từ Orchestrator,
        # hệ thống sẽ TỪ CHỐI leo thang Pro Tiers (Tiết kiệm chi phí).
        # Nó truyền trực tiếp lỗi về lớp Client để người dùng đợi hoặc hệ thống tự cron-job gọi lại.
        logger.error(f"Ingestion bị đình chỉ bởi sự cố hạ tầng AI: {str(e)}")
        raise

```

Mã nguồn triển khai phía trên giải quyết trọn vẹn yêu cầu phân tách logic: LiteLLM là người chịu trách nhiệm cho đường dẫn truyền dữ liệu (Transport Layer), còn hệ thống FANG và ứng dụng nội bộ nắm giữ quyền kiểm soát chất lượng dữ liệu đầu ra và tối ưu chi phí tài nguyên (Business Layer).<sup>1</sup> Sự leo thang cấp độ chỉ xảy ra khi và chỉ khi bộ máy Heuritic của FANG đưa ra kết luận (Verdict) rằng năng lực tư duy của mô hình AI thuộc nhóm Lite là không đủ. Kết quả trả về sau đó tiếp tục bước sang giai đoạn biến đổi từ JSON thành Markdown để giữ kết cấu tiêu đề, phục vụ cho quá trình trích xuất thông tin chung (global\_context) và Phân Mảnh (Hybrid Chunking) tại bước 3 và bước 4.<sup>1</sup> Ở bước 4, mảnh ngữ cảnh chung chứa Tên, Năm Kinh Nghiệm và Kỹ Năng của ứng viên sẽ được "tiêm" thêm vào phần đầu của mỗi khối chunk bị cắt nhỏ.<sup>1</sup>

## 6. Chiến Lược Kỹ Thuật Lớp Nhúng Vector (Embedding Layer) & Chuẩn Hóa Không Gian Đệm (Zero-Padding)

Sau quá trình băm nhỏ văn bản (Chunking), dữ liệu đi vào bước 5: Nhúng Vector (Embedding).<sup>1</sup> Mỗi văn bản con lúc này là một đối tượng ChunkPayload đi kèm bộ nhận diện biến môi trường (gồm thuộc tính content - văn bản, tokenCount - độ dài token, và chunkIndex - thứ tự đoạn).<sup>1</sup> Lớp dịch vụ Nhúng (app/services/embedding.py) tại hệ thống FANG hiện đang vận hành dựa trên một kiến trúc cố định: sử dụng mô hình gốc text-embedding-3-small từ nhà cung cấp OpenAI.<sup>1</sup>

Quá trình cấu hình đã xác định việc nén từ không gian nguyên bản 1536 chiều của mô hình xuống còn mức 1024 chiều là lựa chọn phù hợp trên môi trường thử nghiệm nhằm cân bằng giữa việc giảm kích thước bộ nhớ chỉ mục (HNSW index) trong bảng AIDOCUMENTCHUNK của lược đồ Postgres và việc duy trì hiệu suất tính toán ngữ nghĩa.<sup>1</sup> Đồng thời, kiểu dữ liệu lưu trữ dưới dạng nửa phân giải halfvec(1024) được khai thác tối đa.<sup>1</sup> Biến số EMBEDDING\_VECTOR\_TYPE cho phép nhà quản trị kiểm soát động kích bản thay đổi sang kiểu dữ liệu vector đầy đủ, song song với việc cung cấp khả năng đánh giá điểm chuẩn (benchmark) giữa chi phí và độ chính xác.<sup>1</sup>

Tuy nhiên, định dạng cơ sở dữ liệu khóa tĩnh (Hard-locked DB Schema) tạo ra một điểm mù chỉ mạng: Sự bất đồng bộ không gian (Spatial Dimensionality Mismatch) khi thay đổi nhà cung cấp AI. Nếu trong tương lai, một mô hình giá rẻ hoặc mô hình nguồn mở miễn phí cung cấp các vector ở định dạng 768 chiều (ví dụ: dòng mô hình BERT), cấu trúc cơ sở dữ liệu halfvec(1024) sẽ từ chối truy cập qua một lỗi tràn hệ thống.

Quyết định chiến lược được đánh dấu từ "NOTE FROM Hưng" đã chốt phương án giải quyết bài toán học búa này. Việc sửa đổi mô hình cơ sở dữ liệu (Migration) hoặc viết các kịch bản sao chép lược đồ phức tạp trong một môi trường tài nguyên eo hẹp là rủi ro tuyệt đối không thể chấp nhận được.<sup>1</sup> Giải pháp đưa ra là phải ưu tiên một cơ chế đệm chuẩn hóa tại lớp ứng dụng, lấy 1024 làm thước đo tiêu chuẩn.<sup>1</sup> Hệ thống sẽ thiết kế sử dụng thuật toán Đệm Số Không (Zero-Padding) làm cơ chế phòng thủ tạm thời (Stopgap measure) nhằm cố định kích thước, tạo điều kiện giữ cho đường ống hoạt động độc lập và lên lịch cho việc thiết lập lộ trình nhúng lại toàn bộ các đối tượng lưu trữ (Re-embed) sau này khi năng lực kỹ thuật máy chủ mạnh hơn.<sup>1</sup>

## 6.1. Triển Khai Mã Nguồn Chuẩn Hóa Bằng Numpy

Lớp dịch vụ cấu trúc Nhúng (Embedding) sẽ trực tiếp tích hợp khối tiêu chuẩn LiteLLM cho giao tiếp mạng, sử dụng cấu trúc động từ Nguồn Chân Lý, và bổ sung thêm hàm Toán học xử lý tensor không gian bằng thư viện NumPy cực kỳ hiệu quả.<sup>1</sup>

Python

```
# app/services/embedding.py
import logging
import numpy as np
from typing import List, Any
from litellm import embedding
from app.core.models.registry_schema import RegistryManager

logger = logging.getLogger(__name__)
```

```
class EmbeddingService:
```

```
    """
```

```
    Quản lý quy trình (Pipeline) biến đổi đối tượng ngữ nghĩa thành vector toán học.
```

```
    Đảm bảo 100% tuân thủ lược đồ cấu trúc Postgres pgvector.
```

```
    """
```

```
    def __init__(self):
```

```
        # Truy xuất thông tin cấu hình Nhúng từ Registry
```

```
        self.registry = RegistryManager.get_config()
```

```
        embed_config = self.registry.embedding_models
```

```
        self.target_dim = embed_config.target_dimension # Bắt buộc là 1024
```

```
        # Mặc định gọi model đã quy định
```

```
        default_provider = embed_config.default_provider
```

```
        # Giải nén đường dẫn gọi SDK
```

```
        self.model_path = embed_config.models["text-embedding-3-small"].litellm_path
```

```
    def _normalize_spatial_dimension(self, raw_vector: List[float]) -> List[float]:
```

```
        """
```

```
        Thực thi quyết định kiến trúc cốt lõi: Chuẩn hóa bất đồng bộ kích thước vector.
```

```
        Đảm bảo lược đồ cơ sở dữ liệu halfvec(1024) không bị lỗi.
```

```
        """
```

```
        current_dim = len(raw_vector)
```

```
        # Tối ưu hóa: Nếu đã khớp kích thước, trả về trực tiếp
```

```
        if current_dim == self.target_dim:
```

```
            return raw_vector
```

```
        # Biến đổi thành ma trận Numpy (mảng 1 chiều) để vận hành hàm số
```

```
        vector_np = np.array(raw_vector, dtype=np.float32)
```

```
        if current_dim < self.target_dim:
```

```
            # THUẬT TOÁN ĐỆM SỐ KHÔNG (ZERO-PADDING)
```

```
            # Áp dụng cho các mô hình có số chiều nhỏ (ví dụ: 768).
```

```
            # Bổ sung các giá trị 0 vào phía cuối chiều không gian để lấp đầy khoảng trống.
```

```
            padding_length = self.target_dim - current_dim
```

```
            padded_vector = np.pad(
```

```
                vector_np,
```

```
                (0, padding_length),
```

```
                mode='constant',
```

```
                constant_values=0
```

```
            )
```

```
            logger.warning(f"Chuẩn hóa không gian: Bổ sung không gian từ {current_dim} lên
```

```

{self.target_dim} (Zero-Padding).")
    return padded_vector.tolist()

else:
    # THUẬT TOÁN CẮT CỤT (TRUNCATION)
    # Áp dụng cho mô hình nguyên thủy OpenAI trả về 1536 chiều.
    # Tiến hành loại bỏ các chiều thông tin phía sau.
    truncated_vector = vector_np[:self.target_dim]
    logger.warning(f"Chuẩn hóa không gian: Cắt xén thông tin từ {current_dim} xuống
{self.target_dim} (Truncation).")
    return truncated_vector.tolist()

def generate_embeddings(self, chunks: List[Any]) -> List[List[float]]:
    """
    Ánh xạ danh sách đối tượng ChunkPayload sang hệ vector không gian (Bước 5)
    """
    # Mỗi ChunkPayload bao gồm thuộc tính 'content' đã qua khâu chèn global_context
    texts = [chunk.content for chunk in chunks]

    try:
        # Giao tiếp với nhà cung cấp bằng giao thức đồng nhất LiteLLM
        response = embedding(
            model=self.model_path,
            input=texts
        )

        processed_embeddings =

        # Xử lý theo lô (batch processing) đầu ra và ánh xạ 1-1
        for item in response.data:
            raw_vec = item["embedding"]

            # Đi qua trạm chuẩn hóa an toàn
            normalized_vec = self._normalize_spatial_dimension(raw_vec)
            processed_embeddings.append(normalized_vec)

        return processed_embeddings

    except Exception as e:
        logger.error(f"Sự cố hệ thống Nhúng dữ liệu: {str(e)}")
        raise

```

## 6.2. Phân Tích Đánh Đổi Trong Toán Học Vectơ Và Sự Kiện Đồng Bộ

## Hóa

Biện luận phía sau quyết định toán học này có giá trị cực kỳ lớn về mặt học thuật ứng dụng (applied academic implications).

Thuật toán Chỉ mục HNSW (Hierarchical Navigable Small World) được cấu hình tại Postgres sử dụng khoảng cách Cosine (Cosine Similarity) để xác định biên độ song song giữa hai vector.<sup>1</sup> Công thức tính độ tương đồng bằng thương số giữa Tích vô hướng (Dot Product) và Tích các Độ dài (Magnitude).

Khi hệ thống áp dụng kỹ thuật đệm số 0 (Zero-Padding) cho các mô hình có độ phân giải thấp (như 768 chiều), việc bổ sung 256 con số không ở phần đuôi véc-tơ (tail) thực chất hoàn toàn không làm thay đổi giá trị Tích vô hướng của các hệ tọa độ tương đồng; đồng thời, các giá trị 0 không làm thay đổi Độ lớn tổng thể của véc-tơ ban đầu.<sup>1</sup> Sự dịch chuyển này làm sai lệch nhẹ cấu trúc vi mô phân phối xác suất hình học, nhưng giữ lại được phương hướng nền tảng. Khi cả đoạn văn bản truy vấn (Prompt) và đoạn khối (Chunks) trong DB đều được xử lý qua cùng một khối băng thông bù số lượng này, sự tương đương Cosine được bảo toàn gần như nguyên vẹn, đảm bảo hệ thống tìm kiếm vector ở bước 4 của quy trình truy vấn đàm thoại diễn ra suôn sẻ.<sup>1</sup>

Ngược lại, với kỹ thuật cắt cụt (Truncation) cho không gian 1536 chiều, việc loại bỏ hẳn đi 512 biến không gian ở cuối đồng nghĩa với sự mất mát thông tin thực sự (Data Loss) về các yếu tố ngữ nghĩa thứ cấp. Điểm đánh đổi duy nhất được chấp thuận ở đây là bảo vệ nguyên trạng chi phí phát triển và cấu trúc DB hiện tại. Như một phương án thay thế, hệ thống đã đưa việc "nhúng lại (Re-embed) dữ liệu khi có nguồn lực hạ tầng" thành định hướng chuẩn trong tương lai.<sup>1</sup> Nó hoàn toàn không cản trở việc lưu trữ dữ liệu (Persistence) bước số 6 diễn ra liên tục, phân bổ các bản ghi vector vào bảng AIDOCUMENTCHUNK và bảng lưu nội dung nguyên gốc CVPARSED cho mục đích truy xuất gốc rễ về sau.<sup>1</sup>

## 7. Các Phương Án Quản Lý Lưu Trữ và Theo Dõi Giám Sát Nền Tảng (Telemetry)

Với sự di dời logic giao tiếp sang SDK LiteLLM, tầng Dữ Liệu (Data Layer) tiếp tục vận hành mạnh mẽ trên nền tảng PostgreSQL mà không phải chịu ảnh hưởng do thay đổi khung luồng truy vấn API.<sup>1</sup>

Việc đảm bảo tính ổn định bền vững (Persistence) đối với phiên trò chuyện được điều khiển qua hai tệp `app/services/chat_persistence.py` và `app/services/persistence.py`. Nhóm tệp tin này trực tiếp tham gia quản lý việc nhập lịch sử vào các bảng AICHATCONVERSATION và AICHATMESSAGE, hiện thực hóa kiến trúc quản lý đa kênh lịch sử ATS của FANG.<sup>1</sup>

### 7.1. Bổ Sung Cơ Chế Theo Dõi Nền Tảng

Một tính năng vô cùng đáng giá được cấu hình đi kèm chính là việc LiteLLM tự động cho phép theo dõi chi phí cấp phân luồng (request-level tracking).<sup>1</sup> Nhóm kỹ sư có thể tích hợp một số thư viện theo dõi hiệu năng và ngân sách nền tảng ngay bên trong tệp Cấu hình Khởi động

config.py để ghi log toàn bộ lưu lượng tính bằng USD vào hệ thống giám sát. Cụ thể, các thuộc tính chi phí bao gồm cost\_per\_1m\_input và cost\_per\_1m\_output tại YAML sẽ được cung cấp động để tính hóa đơn tiêu dùng. Khối lượng và chi phí từng phiên đàm thoại (conversation ID) nay đã có cơ sở hạ tầng mạnh mẽ để hỗ trợ minh bạch.<sup>1</sup>

## 7.2. Phương Thức Vận Hành Kịch Bản Thử Nghiệm

Khi tiến hành di dời và hợp nhất mã nguồn, yêu cầu bắt buộc là không được làm hỏng chuỗi thao tác kịch bản đã được định hình tại hệ thống v2.<sup>1</sup> Do mọi tệp mã nguồn nghiệp vụ import openai hoặc import google.generativeai đều đã bị gỡ bỏ trong nhóm lớp Dịch Vụ, đội ngũ kỹ sư sẽ kiểm nghiệm trạng thái hệ thống sử dụng các đường truyền kịch bản khói (smoke tests) điển hình.

Quá trình chạy lệnh thực thi python smoke\_tests/test\_e2e\_pipeline.py (chạy trên toàn bộ luồng pipeline) và kịch bản test\_parser.py (kịch bản chỉ dành riêng đánh giá CV) phải được tiến hành bằng việc đưa các mã API Khóa (API keys) như OPENAI\_API\_KEY, GEMINI\_API\_KEY và ANTHROPIC\_API\_KEY vào tệp .env môi trường của lõi máy chủ FANG. Thông qua SDK tích hợp, tiến trình phân tích tự động (parser) có thể tiếp cận mọi dịch vụ trung tâm thông qua các kết nối được ủy quyền (Delegation Connection), không đòi hỏi sự rườm rà. Nếu hệ thống cơ sở dữ liệu vô tình thay đổi kích thước vector do thay đổi nhà cung cấp tại cấu hình, kịch bản test\_chunking.py và test\_parser\_db.py sẽ bảo vệ hệ thống khỏi ngoại lệ sụp đổ, cung cấp một khoảng thời gian trống cho phép xử lý và dọn dẹp môi trường thử nghiệm cục bộ thông qua khối lệnh scripts/reset\_and\_seed\_db.py nhằm tới micareer\_lite\_db.

## 8. Kết Luận Nâng Cấp Hệ Thống Toàn Diện

Phân tích định hướng thiết kế cấu trúc mạng lưới dịch vụ thông minh (AI Services) cho nền tảng FANG FastAPI đã trình bày một lộ trình tiến hóa sâu sắc và dứt khoát. Việc loại bỏ các bộ điều hợp thủ công (manual adapter chains) phức tạp để tích hợp giải pháp ủy quyền mạng lưới proxy cục bộ từ LiteLLM SDK tạo ra một sự thay đổi mô hình thiết kế quan trọng từ mã hóa cứng sang tiếp cận khai báo (declarative approach).<sup>1</sup> Bằng cách di dời cấu hình, quyết định leo thang (escalation matrix), chỉ số token và chuỗi dự phòng sang một tệp "Nguồn chân lý duy nhất" models\_registry.yaml, lớp xử lý AI Core (lõi FANG) đã đạt được khả năng độc lập hệ thống toàn diện, bất khả tri về mô hình mạng và tối đa hóa khả năng vận hành.<sup>1</sup>

Những quyết định về chiến lược quản lý xử lý lỗi dựa trên mạng phân tầng đa cấp (5-Tier parser & Fallback strategy) giữ vai trò như một màng lọc tối ưu, hạn chế hiện tượng tràn lưu lượng tài chính (Cost Spillover), sử dụng các mô hình cấp thấp làm đệm trước khi điều động tài nguyên Pro Tiers phức tạp.<sup>1</sup> Song song với đó, nguyên lý đảm bảo không can thiệp cơ sở dữ liệu của hạ tầng đang chạy, vốn được phê chuẩn ở giai đoạn trước, được bảo vệ thành công bởi các thuật toán nhồi số không và cắt cụt toán học tensor tại bước nhúng (Embedding zero-padding), biến nhược điểm giới hạn lưu trữ dữ liệu halfvec trở thành một chốt chặn chống gián đoạn ổn định.<sup>1</sup> Nhìn chung, kết cấu kiến trúc đề xuất giải quyết bài toán giảm thiểu nợ kỹ thuật (Technical Debt Reduction) triệt để và đặt nền tảng sẵn sàng cho sự thay đổi mạnh mẽ của các thế hệ LLM và

hệ thống nhúng trong tương lai mà không ảnh hưởng tới trải nghiệm người dùng cuối ở nhánh Thin Client.

### **Nguồn trích dẫn**

1. embedding\_strategy.md